PRINCIPIOS DE PROGRAMAÇÂO PROCEDIMENTAL

MANUAL DE PROGRAMADOR

Professor: Nuno Pimenta

Realizado por:

António Manuel Marinho Vicente 2023225326

Miguel Alexandre Pontes Calado 2022228235

INTRODUÇÂO

No âmbito da cadeira de Princípios de Programação Procedimental foi-nos proposto a realização de um projeto que consiste no desenvolvimento de uma aplicação para gerir os doentes de um médico. Para realizar estre projeto criámos 3 ficheiros com código fonte, o projeto.c que corresponde ao main com o menu, o projeto_funcoes.c que corresponde a todas as funções que utilizámos ao longo do projeto e o projeto_struct.h que corresponde ao conjunto de structs que vão ser utilizadas.

Ficheiro projeto.c

Neste ficheiro temos o main que vai ter um menu para o utilizador escolher a opção que deseja realizar:

- Realizámos o include do ficheiro projeto_struct.h e de algumas bibliotecas necessárias para o programa
- De seguida temos a função main onde criamos a listaDoentes usando a função cria_listaDoentes() e inserimos os doentes e os registos guardados nos ficheiros doentes.txt e registos.txt usando a função insere_doentes_do_ficheiro()
- Inicializámos a variável opcao com 0 e inciciámos um ciclo while que só termina quando o valor da variável opção for 7.
- Utilizámos um switch-case para que em cada número que selecionamos vai ser chamada a função específica e executar o seu código.
- O menu vai ser exibido ao utilizador logo no início da execução com várias opções de 1 a 7. Assim que o número é inserido na consola a sua funcionalidade é executada e o programa retorna ao menu permitindo ao utilizador escolher outra funcionalidade para executar.

Ficheiro projeto_struct.h

Este ficheiro é utilizado principalmente como cabeçalho, neste são declaradas estruturas de dados e os nomes das funções que são utilizadas no projeto. Assim, permite que outros ficheiros fonte incluam este cabeçalho e utilizem as estruturas e as funções nele contidos.

Definimos diversas estruturas tais como:

- “Data” que possui os campos “dia”, “mes” e “ano” que vão respresentar uma data.
- “Doente” que possui os campos “nome” para armazenar o nome , “nascimento” do tipo “Data” para armazenar a data de nascimento do doente, “id” para armazenar o id do doente, “cc” para armazenar o numero cartão de cidadão do doente, “telefone” para armazenar o telefone do doente, “email” para armazenar o email do doente e um ponteiro para a “lista_dados” do tipo “ListaDados”.
- “noLista” que possui a variável doente do tipo “Doente” e um ponteiro “prox” pra o próximo nó da lista
- “NoRegisto” que possui uma variável registo do tipo “RegistoDados” e um ponteiro prox para o próximo registo
- “ListaDados” que possui um ponteiro para o início, outro para o fim e outro para o próximo nó da lista
- “RegistoDados” que possui os campos “data” do tipo ” Data” para armazenar a data, “ta_min” para armazenar a tensão mínima, “ta_max” para armazenar a tensão máxima, ”peso” para armazenar o peso e “altura” para armazenar a altura

Ficheiro projeto_funcoes.c

void insere_doente(pLista lista, Doente d1) :
Esta função é uma função auxiliar, para otimizar o código. A função começa por percorrer a lista que lhe é dada, á procura do "lugar" do doente (d1) que é também fornecido, como é pretendida a ordenação por ordem alfabética, nós usámos a função strcasesmp(), esta compara todos os caracteres de uma string e retorna (-1,0,1) de acordo com a sua posição, ou seja, -1 caso a string esteja posicionada antes da outra string, 0 caso sejam iguais, ou 1, caso esteja posicionado à frente da outra string, tendo em conta a ordem alfabética. Quando a sua posição é encontrada, inserimos o respetivo doente na posição encontrada.

pLista procurar_pessoa_por_id(pLista lista, int id):
Nesta função, tal como diz o nome iremos procurar o doente através do seu id. Utiliza-se o ciclo while para percorrer a lista, e á medida que o fazemos comparamos o id que é fornecido à função com o id de cada doente presente na lista, caso os id's sejam iguais, a função retorna o doente com o tal id. Caso a pessoa com esse id não seja encontrada a função retorna NULL.

void insere_doentes_do_ficheiro(pLista lista, char *nome_ficheiro, char* ficheiro_registos) :
Esta função começa por percorrer o ficheiro que contém os dados do doente, e atribuindo os mesmos a uma variável temporária, em seguida damos uso á função "insere_doente" para inserir o respetivo doente no seu lugar, na ordem alfabética, irá repetir-se até chegar ao fim do ficheiro. Faz o mesmo para o segundo ficheiro, só que neste caso não usámos uma função auxiliar para inserir o registo. Como nós optamos por inserir as tensões por ordem decrescente de tensão máxima, iremos inicialmente confirmar se existe algum registo para o doente, se não existir o registro será posicionado no início, caso contrário iremos procurar o seu lugar de modo a que respeite a ordem.

void guarda_no_ficheiro(pLista lista) :
Esta função é utilizada para guardar os dados e os registos nos respetivos ficheiros, esta apenas é chamada quando fechamos o programa. Nesta função percorremos a lista dados e a lista de doentes e dentro de si percorremos as informações de cada doente inserindo-os de volta nos ficheiros.

void listar_doentes_ta(pLista lista) :
Com esta função iremos listar todas as tensões máximas acima de certo valor, valor esse dado pelo utilizador. Basicamente percorremos a lista_dados contendo os registos, com o propósito de comparar todas as tensões de todos os doentes, com a tensão dada de referência pelo user.

int existe_cc(pLista lista, int cc):
Nesta função iremos verificar se existe tal número de cartão de cidadão (int cc) na função, esta serve de auxílio a outra função.

int id_maxi(pLista lista):
Mais uma função auxiliar, com esta iremos verificar qual o id máximo presente na lista de doentes, com o propósito de saber qual será o próximo id.

void menu_add_doente(pLista lista) :
Esta função será responsável pelo registo das informações do doente (cc,telefone,email,nome e data de nascimento), o código é composto maioritariamente pelas chamadas "barreiras" do utilizador, ou seja barreiras que não permitam a inserção de números de telefone com letras, nomes com números, ou emails sem "@", também colocamos um limite de números, para não serem inseridos nomes demasiado grandes, ou nomes demasiado longos. Nesta função damos uso à função "id_maxi" para determinar o id que será atribuído ao doente em questão.

void remover_doente(pLista lista):
Para remover um doente da lista usaremos esta função, inicialmente confirmamos se o número de cartão de cidadão fornecido pelo utilizador, existe na lista, caso exista iremos remover os dados do doente da lista, e iremos atualizar os ponteiros da lista.

void imprime_listaDoentes (pLista lista):
Esta função será responsável por imprimir as informações de todos os doentes.

pLista procurar_pessoa_por_cc(pLista lista, int cc) :
Esta função é muito similar á função procurar_pessoa_por_id, só que envés de o termo de comparação ser o id, neste caso será o número de cartão de cidadão(cc). Esta é também uma função auxiliar.

void registar_dados_doente(pLista lista):
Nesta iremos fazer o registo dos dados do doente, tal como tensão (máxima e mínima), peso, altura, e data quando se fez o registo. Começamos por pedir o cc e nome do doente, e em seguida registarmos todos os dados num novo registo, neste processo tivemos o cuidado de colocar as barreiras para o utilizador. Em seguida iremos procurar o lugar deste registo, por entre os outros registos do doente em questão, caso estes existam, e iremos o posicionar corretamente de acordo com a ordem decrescente de tensões máximas, semelhante ao que fizemos na função "insere_doentes_do_ficheiro" (na parte referente aos registos)

void imprimir_dados_doente(pLista lista):
Por fim, esta função irá imprimir todos os registos de um doente. É pedido ao utilizador o cc do doente em questão, com este dado fornecido, serão impressos primeiramente os dados do doente no que toca a nome, telefone, id e data de nascimento, e em seguida todos os registos que o doente tem no seu nome, previamente ordenados.